Herald

Every alert reaches the right destination — no command syntax required.

Summary

Herald ingests system events and reliably fans them out to multiple notification channels so every alert reaches the right place. Subscribers interact in plain natural language; Herald infers intent via a three-tier discipline (command fast-path → LLM intent inference → clarify fallback).

Short description

An event ingestion and multi-channel notification fan-out system. Herald reliably routes system events to the right destinations across messenger channels, and lets subscribers speak plain natural language — resolving intent through a command fast-path, LLM inference, and a clarify-and-ask fallback.

Long description

Herald is the notification backbone that guarantees a system event actually lands where a human can act on it — the unglamorous but mission-critical layer where most homegrown alerting quietly fails. It ingests events and fans them out reliably across multiple notification channels, closing off the familiar failure modes where an alert is dropped, mis-routed to a dead channel, or buried under noise until it’s too late to matter. But reliable delivery is only half the story; the other half is what happens when a human wants to respond. Here Herald refuses the usual bargain where users must memorize a rigid command syntax to interact with an alerting bot. Subscribers simply write in plain natural language, and Herald resolves what they meant through a deliberate three-tier discipline: a fast-path that recognizes explicit commands instantly, then LLM-based intent inference (via Claude Code) for free-form messages, and finally a `clarify` fallback that replies, tags, and asks a question when intent

is genuinely ambiguous. That “recognize → infer → clarify” ladder is the whole design philosophy in miniature — the common case stays instant and deterministic, the flexible case is handled by a model, and the uncertain case is never resolved by a blind guess that fires the wrong action. Herald also models participation and attribution: an operator-username env var (`HERALD_<CHANNEL>_OPERATOR_USERNAME`) and a participant/attribution contract drive `created_by`/`assigned_to` fields and notification @-tagging, so it is clear who did what and who is being notified. Governance-wise, Herald inherits the Helix Constitution as a co-located submodule and follows its rules, and it is an early production consumer of Docs Chain — its full 66-document Markdown→HTML/PDF/DOCX corpus is wired through Docs Chain `exec:` transforms and verifies clean. Herald is primarily Shell/Go tooling with layered specifications (V1→V2→V3→V4 supersession) and per-channel operator setup guides for messengers and LLM/agent dispatchers.

Why we built it

Alerts fail quietly — sent to the wrong channel, dropped, or requiring rigid command syntax users won’t remember. Herald was built to guarantee reliable fan-out and to let people respond in natural language, so notifications are both dependable and effortless to act on.

Why it’s a game-changer

It fuses two things that are usually bought as separate products — dependable multi-channel event routing and a natural-language interface — into one system where operators simply talk and the software works out what they mean. The clarify fallback is the detail that makes it trustworthy in production: an alerting system that would rather ask than misfire is one you can actually let touch real state.

What’s innovative

- Three-tier intent discipline: command fast-path → LLM inference → clarify-and-ask.
- Natural-language subscriber interaction (no command syntax to learn).
- Participant-attribution contract driving `created_by`/`assigned_to` + @-tagging.
- Real Docs Chain consumer (66-doc corpus, multi-format, verified).

Challenges & solutions

- **Ambiguous natural-language intent:** solved with the three-tier recognize/infer/clarify ladder rather than blind guessing.
- **Reliable fan-out:** solved with an ingestion→multi-channel dispatch design so alerts reach the correct destination.
- **Correct attribution across channels:** solved with the operator-username env var and participant-attribution contract.
- **Documentation drift:** solved by wiring its docs corpus through Docs Chain with verified transforms.

Tech stack (why + how)

- **Go** — core event/dispatch logic (per org language patterns).
- **Shell** — operator tooling and setup scripts.
- **Claude Code (LLM)** — intent inference tier for free-form messages.
- **Messenger channel adapters** — multi-channel notification fan-out.
- **Docs Chain** — documentation build/verify pipeline (Markdown→HTML/PDF/DOCX).
- **Helix Constitution submodule** — inherited governance/rules.